

Chương 1: Máy Tính, Bộ Nhớ RAM, CPU & Bản Chất Thuật Toán

[THUẬT NGỮ KHOA HỌC] Thuật Toán (Algorithm)

Từ nguyên học: Bắt nguồn từ tên nhà toán học Ba Tư thế kỷ 9 Muhammad ibn Musa al-Khwarizmi, người tiên phong trong việc chuẩn hóa các quy tắc tính toán số học.

Mô hình trực giác: Một chuỗi hữu hạn các chỉ thị logic rõ ràng, tất định và có thể thực thi từng bước trên giấy mà không cần phỏng đoán, nhằm biến đổi dữ liệu đầu vào (Input) thành đầu ra (Output) mong muốn.

1.1 Kiến Trúc Máy Tính & Chu Trình CPU

Mọi máy tính hiện đại đều vận hành dựa trên kiến trúc Von Neumann với hai thành phần cốt lõi:

- Bộ xử lý trung tâm (CPU - Central Processing Unit):** Đóng vai trò là “bộ não” thực thi các chỉ thị số học và logic cơ bản (+, -, so sánh, nhảy có điều kiện). CPU chứa các ngăn chứa dữ liệu siêu tốc độ gọi là **thanh ghi (registers)**.
- Bộ nhớ truy xuất ngẫu nhiên (RAM - Random Access Memory):** Là một kho chứa khổng lồ các ô nhớ nhị phân lưu trữ chỉ thị chương trình và dữ liệu đang được xử lý.

CPU vận hành tuần tự theo chu kỳ ba pha liên tục:

Fetch (Nạp chỉ thị) → Decode (Giải mã) → Execute (Thực thi & Ghi nhận kết quả)

[TRỰC GIÁC HÌNH TƯỢNG] Mô hình dải băng giấy của RAM

Hãy tưởng tượng bộ nhớ RAM như một thước dây dài vô tận được chia thành các ô vuông liên tiếp. Mỗi ô vuông tương ứng với một **Byte** (gồm 8 bit nhị phân). Mỗi Byte được định danh duy nhất bằng một con số nguyên gọi là **Địa chỉ ô nhớ (Memory Address)**, bắt đầu từ chỉ số 0, 1, 2, 3, Khi CPU cần đọc hoặc ghi vào ô nhớ nào, nó chỉ cần phát tín hiệu địa chỉ đó lên bus hệ thống.

1.2 Biến Số Là Gì? Bản Chất “Nhãn Dán” Lên Ô Nhớ

Khi chúng ta viết trong mã nguồn:

```
int a = 5;
```

Trình biên dịch thực chất làm hai việc:

- Xin hệ điều hành cấp phát một khoảng ô nhớ liên tiếp trong RAM (đối với kiểu `int` 32-bit là 4 bytes).
- Gắn nhãn tên `a` vào địa chỉ đầu tiên của vùng nhớ đó.

Do đó, **biến không phải là một chiếc hộp thần kỳ**. Biến đơn thuần là một cái tên thân thiện với con người giúp lập trình viên không phải nhớ những con số địa chỉ phần cứng trần trụi như `0x7ffee4`.

[MÔ XẼ CÚ PHÁP C++14]	
<code>int a = 5; int b = 7; int temp = a; a = b; b = temp;</code>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>int a = 5;</code>	Cấp phát 4 bytes cho biến a và nạp giá trị nhị phân của 5 vào vùng nhớ.
<code>int b = 7;</code>	Cấp phát 4 bytes tiếp theo cho biến b và nạp giá trị 7.
<code>int temp = a;</code>	Đọc giá trị từ ô nhớ a (5) và ghi vào ô nhớ tạm thời temp.
<code>a = b;</code>	Đọc giá trị từ ô nhớ b (7) và ghi đè vào ô nhớ a (giá trị cũ 5 bị xóa sổ).
<code>b = temp;</code>	Đọc giá trị từ ô nhớ temp (5) và ghi đè vào ô nhớ b hoàn tất hoán đổi.

1.3 Quy Trình Chạy Bàn (Hand-trace) Trạng Thái Ô Nhớ Trên Giấy

Khi học lập trình thi đấu không cần máy tính, công cụ mạnh nhất của bạn chính là một mẫu giấy nháp và chiếc bút chì. Hãy vẽ một bảng theo dõi trạng thái các ô nhớ qua từng dòng lệnh:

Bước	Dòng lệnh	Ô nhớ a	Ô nhớ b	Ô nhớ temp
0	Khởi tạo ban đầu	3	5	[chưa tạo]
1	<code>int temp = a;</code>	3	5	3
2	<code>a = b;</code>	5	5	3
3	<code>b = temp;</code>	5	3	3
Kết quả	Hoàn tất hoán đổi	5	3	3

1.4 Bài Tập Tiêu Chuẩn

Bài toán: Bài 1.1 - Hoán Đổi Ô Nhớ RAM (RAM Memory Swap) (Nguồn: Nền tảng C++14 - Thao tác bộ nhớ)

Cho hai số nguyên A và B nạp vào hai biến ban đầu `a` và `b`. Em hãy thực hiện thuật toán hoán đổi giá trị của `a` và `b` thông qua biến trung gian `temp`, đồng thời tính tổng $S = A + B$.

Đầu vào (Input): Hai số nguyên A, B trên cùng một dòng ($-100 \leq A, B \leq 100$).

Đầu ra (Output): In ra 3 số nguyên A', B', S cách nhau một khoảng trắng, trong đó A', B' là giá trị của `a` và `b` sau khi hoán đổi.

Ví dụ mẫu:

Đầu vào (Input):

Đầu ra (Output):

35

538

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì):

Đáp án đối chiếu xem tại Phụ lục cuối sách

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	35	[.....]
2	Warm-up	1020	[.....]
3	Warm-up	11	[.....]
4	Edge case	00	[.....]
5	Edge case	07	[.....]
6	Edge case	-55	[.....]
7	Trick case	-10-20	[.....]
8	Trick case	-80	[.....]
9	Mental challenge	4217	[.....]
10	Mental challenge	-1535	[.....]

Chương 2: Cấu Trúc Chương Trình C++14 & Biểu Diễn Dữ Liệu

[THUẬT NGỮ KHOA HỌC] Bù Hai (Two's Complement)

Từ nguyên học: Được nhà toán học John von Neumann đề xuất năm 1945 trong tài liệu EDVAC để đơn giản hóa phần cứng số học trong máy tính điện tử.

Mô hình trực giác: Một quy ước biểu diễn số nguyên có dấu bằng hệ nhị phân, trong đó bit có trọng số lớn nhất (MSB) đóng vai trò là bit dấu mang trọng số âm -2^{k-1} . Quy ước này cho phép mạch cộng xử lý phép trừ $A - B$ hệt như phép cộng $A + (-B)$.

2.1 Cấu Trúc Chương Trình C++14 Chuẩn Mục

Một chương trình C++14 bắt đầu với hàm `main()` - điểm khởi đầu thực thi của hệ điều hành:

```
#include <iostream>

int main() {
    // Thân chương trình đặt trong cặp ngoặc nhọn
    std::cout << "Akashic Record C++14\n";
    return 0; // Trả về mã lỗi 0 báo hiệu chương trình kết thúc thành công
}
```

2.2 Các Kiểu Dữ Liệu Nguyên Thủy & Kích Thước Ô Nhớ

Kiểu dữ liệu	Kích thước	Dải giá trị	Ứng dụng trong Lập trình thi đấu
<code>bool</code>	1 Byte	0 (<code>false</code>) hoặc 1 (<code>true</code>)	Lưu cờ trạng thái, kiểm tra điều kiện
<code>char</code>	1 Byte	-128...127 (ASCII 0...127)	Lưu ký tự chữ số, ký tự văn bản
<code>int</code>	4 Bytes	$-2^{31} \dots 2^{31} - 1 \approx \pm 2 \times 10^9$	Chỉ số mảng, đếm, tính toán số học thông thường
<code>long</code> <code>long</code>	8 Bytes	$-2^{63} \dots 2^{63} - 1 \approx \pm 9 \times 10^{18}$	Tổng tích lớn, số lượng cấu hình tổ hợp
<code>double</code>	8 Bytes	Độ chính xác $\approx 15 - 17$ chữ số thập phân	Số thực, hình học giải tích, xác suất

2.3 Cơ Chế Bù Hai & Vòng Tròn Tràn Số Nguyên (Integer Overflow)

Để lấy số đối của một số nhị phân trong hệ bù hai, ta đảo tất cả các bit ($0 \rightarrow 1, 1 \rightarrow 0$) rồi cộng thêm 1:

$$-x = (\sim x) + 1$$

[LƯU Ý QUAN TRỌNG] Cạm bẫy tràn số nguyên nguy hiểm nhất

Trong kiểu số nguyên 32-bit có dấu `int`, giá trị lớn nhất có thể lưu được là:

$$2^{\{31\}} - 1 = 2,147,483,647$$

Nếu bạn thực hiện phép tính $2,147,483,647 + 1$, phần cứng CPU sẽ cộng bit và bật bit dấu MSB lên 1, biến kết quả thành con số âm cực đại:

$$2,147,483,647 + 1 = -2,147,483,648$$

Hiện tượng này gọi là **Tràn số nguyên (Integer Overflow)**. Trong thi đấu, một phép nhân giữa hai số $A, B \approx 10^5$ sẽ tạo ra tích $10^{\{10\}} > 2 \times 10^9$, dẫn đến kết quả sai hoàn toàn (Wrong Answer) nếu không ép kiểu!

[MỔ XẺ CÚ PHÁP C++14]	
<code>long long prod = static_cast<long long>(a) * b;</code>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>static_cast<long long>(a)</code>	Ép kiểu tường minh biến 32-bit `a` lên kiểu 64-bit `long long` TRƯỚC KHI thực hiện phép nhân.
<code>* b</code>	Do một toán hạng là `long long`, toán hạng `b` tự động được thăng cấp lên `long long`, phép nhân thực hiện an toàn trong thanh ghi 64-bit.
<code>long long prod =</code>	Ghi kết quả chính xác không bao giờ tràn vào biến 64-bit `prod`.

2.4 Mô Phỏng Vòng Tròn Tràn Số 4-Bit Trên Giấy

Hệ 4-bit bù hai biểu diễn các số từ -8 đến $+7$:

Mã nhị phân 4-bit	Giá trị có dấu (Bù 2)	Giá trị không dấu	Ghi chú
0110	+6	6	Số dương
0111	+7	7	Giá trị dương cực đại của 4-bit
1000	-8	8	Tràn số: $+7 + 1$ lộn vòng sang -8
1001	-7	9	<code>-8 + 1 = -7</code>
1111	-1	15	Tất cả các bit đều là 1

2.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 2.1 - Tích An Toàn & Ngưỡng Giới Hạn
(Safe Product)

(Nguồn: Nền tảng C++14 - Biểu diễn dữ liệu & Tràn số)

Cho hai số nguyên A, B và một ngưỡng kiểm soát M . Em hãy tính tích $P = A \times B$ bằng phép toán ép kiểu an toàn `static_cast<long long>(a) * b`.

Nếu độ lớn tuyệt đối của tích vượt quá ngưỡng M ($|P| > M$), hãy in ra nhãn `OVERFLOW` kèm theo giá trị P . Ngược lại nếu $|P| \leq M$, in ra nhãn `SAFE` kèm theo giá trị P .

Đầu vào (Input): Ba số nguyên A, B, M trên cùng một dòng.

Đầu ra (Output): Chuỗi kết quả gồm nhãn (`SAFE` hoặc `OVERFLOW`) và giá trị tích P cách nhau dấu cách.

Ví dụ mẫu:

Đầu vào (Input):

Đầu ra (Output):

3 4 50

SAFE 12

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	3 4 50	[.....]
2	Warm-up	5 6 20	[.....]
3	Warm-up	2 10 20	[.....]
4	Edge case	0 100 50	[.....]
5	Edge case	10 0 0	[.....]
6	Edge case	-5 6 35	[.....]
7	Trick case	-5 6 25	[.....]
8	Trick case	-4 -7 25	[.....]
9	Mental challenge	12 8 100	[.....]
10	Mental challenge	-15 7 100	[.....]

Chương 3: Biến, Biểu Thức, Ép Kiểu & Luồng Điều Khiển

[THUẬT NGỮ KHOA HỌC] Bất Biến Vòng Lặp (Loop Invariant)

Từ nguyên học: Được nhà khoa học máy tính Tony Hoare đề xuất năm 1969 trong hệ logic Hoare chứng minh tính đúng đắn của chương trình.

Mô hình trực giác: Một khẳng định logic về trạng thái của các biến luôn luôn ĐÚNG trước khi bắt đầu vòng lặp, được duy trì ĐÚNG sau mỗi bước lặp, và đảm bảo tính đúng đắn của kết quả khi vòng lặp kết thúc.

3.1 Toán Tử Số Học & Phép Chia Nguyên Trong C++14

Trong C++14, các phép toán số học cơ bản tuân thủ quy tắc chặt chẽ:

- Phép chia hai số nguyên $\frac{A}{B}$ là phép chia cắt cụt về phía số 0 (truncation towards zero):

$$\frac{7}{2} = 3, \quad -\frac{7}{2} = -3$$

- Phép chia dư $A\%B$:

$$A = \left(\frac{A}{B}\right) \times B + (A\%B)$$

Dấu của kết quả phép dư trong C++ luôn cùng dấu với số bị chia A :

$$7\%3 = 1, \quad -7\%3 = -1$$

3.2 Ép Kiểu Tường Minh `static_cast`

C++14 cung cấp cơ chế ép kiểu an toàn tại thời điểm biên dịch:

```
double avg = static_cast<double>(sum) / n;
```

Trình biên dịch sẽ kiểm tra xem kiểu nguồn có thể chuyển đổi hợp lệ sang kiểu đích hay không, loại trừ hoàn toàn các lỗi ép kiểu con trỏ nguy hiểm của cú pháp C-style cổ điển `((double)sum)`.

3.3 Cấu Trúc Vòng Lặp & Bảng Biến Thiên Trên Giấy

[MÔ XẼ CÚ PHÁP C++14]	
<pre>for (int i = 1; i ≤ n; ++i) { sum += i; }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<pre>int i = 1;</pre>	Khởi tạo biến đếm `i`, chỉ thực thi duy nhất 1 lần khi bắt đầu vòng lặp.
<pre>i ≤ n;</pre>	Điều kiện tiếp tục lặp, được đánh giá TRƯỚC mỗi lần thực thi thân vòng lặp.
<pre>++i</pre>	Bước nhảy, thực thi SAU KHI thân vòng lặp hoàn thành mỗi bước.
<pre>{ sum += i; }</pre>	Thân vòng lặp thực hiện công việc tính toán chính.

3.4 Kỹ Thuật Vẽ Bảng Biến Thiên Vòng Lặp Collatz

Dãy Collatz (còn gọi là giả thuyết $3N + 1$) bắt đầu từ một số nguyên dương N :

- Nếu N chẵn: $N \rightarrow \frac{N}{2}$
- Nếu N lẻ: $N \rightarrow 3N + 1$

Hãy theo dõi bảng biến thiên trên giấy khi khởi đầu với $N = 3$:

Bước	N hiện tại	Điều kiện chẵn/lẻ	Phép biến đổi	N kế tiếp	Max đạt được
0	3	Lẻ	$3 * 3 + 1$	10	10
1	10	Chẵn	$10 / 2$	5	10
2	5	Lẻ	$3 * 5 + 1$	16	16
3	16	Chẵn	$16 / 2$	8	16
4	8	Chẵn	$8 / 2$	4	16
5	4	Chẵn	$4 / 2$	2	16
6	2	Chẵn	$2 / 2$	1	16
7	1	Dừng	Chạm N = 1	1	16

3.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 3.1 - Bảng Biến Thiên Dãy Collatz
(Collatz Sequence Trace)

(Nguồn: Nền tảng C++14 - Vòng lặp & Bảng biến thiên)

Cho một số nguyên dương N . Em hãy thực hiện mô phỏng thuật toán Collatz trên giấy:

- Nếu $N > 1$: tăng số bước lặp `steps`. Nếu N chẵn, gán $N = \frac{N}{2}$; nếu N lẻ, gán $N = 3N + 1$.
- Luôn duy trì và cập nhật giá trị cực đại `max_val` mà biến N đạt được trong suốt quá trình.
- Quá trình dừng lại ngay khi $N = 1$. (Quy ước nếu ban đầu $N = 1$, số bước là 0 và max là 1).

Đầu vào (Input): Một số nguyên dương N ($1 \leq N \leq 30$).

Đầu ra (Output): Hai số nguyên `steps` và `max_val` cách nhau dấu cách.

Ví dụ mẫu:

Đầu vào (Input):

Đầu ra (Output):

3

7 16

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	1	[.....]
2	Warm-up	2	[.....]
3	Warm-up	4	[.....]
4	Edge case	8	[.....]
5	Edge case	3	[.....]
6	Edge case	5	[.....]
7	Trick case	6	[.....]
8	Trick case	7	[.....]
9	Mental challenge	9	[.....]
10	Mental challenge	10	[.....]

Chương 4: Mảng 1D, 2D & Chuỗi Ký Tự Cơ Bản

[THUẬT NGỮ KHOA HỌC] Chỉ Số 0-Based (Zero-based Indexing)

Từ nguyên học: Xuất hiện từ ngôn ngữ lập trình BCPL và C vào thập niên 1960-1970 do nhà khoa học máy tính Martin Richards và Dennis Ritchie phát triển.

Mô hình trực giác: Chỉ số i của mảng không đại diện cho 'thứ tự phần tử' theo cách đếm thông thường của con người, mà đại diện cho 'độ dời khoảng cách' (offset) tính từ vị trí ô nhớ đầu tiên của mảng trên thanh RAM. Phần tử đầu tiên có khoảng cách dời bằng 0, do đó có chỉ số là 0.

4.1 Bản Chất Ô Nhớ Liên Tiếp Của Mảng 1D

Khi khai báo `int a[5]`, hệ thống cấp phát chính xác $5 \times 4 = 20$ bytes nằm liền kề nhau trên RAM:

$$\text{Địa chỉ của } a[i] = \text{Địa chỉ cơ sở (base)} + i \times \text{sizeof(int)}$$

[MÔ HÌNH BỘ NHỚ] Mô hình lát cắt bộ nhớ của Mảng 1D

Giả sử mảng `a` được cấp phát bắt đầu từ địa chỉ `0x100`:

- `a[0]`: Chiếm các byte `0x100 .. 0x103` (Giá trị phần tử đầu tiên)
- `a[1]`: Chiếm các byte `0x104 .. 0x107` (Giá trị phần tử thứ hai)
- `a[2]`: Chiếm các byte `0x108 .. 0x10B` (Giá trị phần tử thứ ba)

Nhờ cấu trúc liên tiếp này, CPU có thể truy xuất bất kỳ phần tử `a[i]` nào trong thời gian tức thì $O(1)$ chỉ bằng một phép tính số học đơn giản!

4.2 Mảng 2 Chiều & Quy Ước Thứ Tự Dòng-Trước (Row-Major Order)

Bộ nhớ RAM là một đường thẳng 1 chiều. Do đó, ma trận $R \times C$ được trải phẳng lần lượt từng hàng:

$$\text{Địa chỉ của } a[i][j] = \text{base} + (i \times C + j) \times \text{sizeof}(\text{kiểu dữ liệu})$$

4.3 Chuỗi Ký Tự `std::string` & Bảng Mã ASCII

Mỗi ký tự `char` thực chất là một số nguyên 8-bit trong bảng mã ASCII:

- Ký tự chữ số `'0'` đến `'9'`: Có mã 48...57. Chuyển đổi: `digit = c - '0'`.
- Chữ hoa `'A'` đến `'Z'`: Mã 65...90.
- Chữ thường `'a'` đến `'z'`: Mã 97...122.
- Khoảng cách giữa chữ thường và chữ hoa: `'a' - 'A' = 32`.

[MỖ XẺ CÚ PHÁP C++14]	
<pre>for (int i = 0; i < len / 2; ++i) { if (s[i] != s[len - 1 - i]) return false; }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>int i = 0; i < len / 2;</code>	Chỉ cần duyệt nửa đầu chuỗi vì mỗi ký tự sẽ được so khớp với ký tự đối xứng ở nửa sau.
<code>s[i]</code>	Ký tự thứ <code>i</code> tính từ đầu chuỗi (bên trái).
<code>s[len - 1 - i]</code>	Ký tự đối xứng tương ứng tính từ cuối chuỗi (bên phải).
<code>return false;</code>	Nếu phát hiện bất kỳ cặp ký tự đối xứng nào khác nhau, chuỗi lập tức không phải Palindrome.

4.4 Quy Trình Kiểm Tra Đối Xứng (Palindrome) Trên Giấy

Với chuỗi $S = \text{radar}$ có độ dài $\text{len} = 5$:

Bước	Chỉ số i	$s[i]$	Chỉ số đối xứng ($4 - i$)	$s[4 - i]$	So khớp
1	0	'r'	4	'r'	Bằng nhau ('r' == 'r')
2	1	'a'	3	'a'	Bằng nhau ('a' == 'a')
Kết luận	-	-	-	-	Chuỗi đối xứng (YES)

4.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 4.1 - Thống Kê Mảng & Chuỗi Đối Xứng (Array Stats & Palindrome)

(Nguồn: Nền tảng C++14 - Mảng 1D & Xử lý Chuỗi)

Cho một mảng A gồm N số nguyên ($1 \leq N \leq 8$) và một chuỗi ký tự S gồm các chữ cái tiếng Anh in thường.

Em hãy thực hiện:

- Tính giá trị trung bình cộng số học $\text{avg} = \sum \frac{A}{N}$ (theo giá trị thực chính xác). Đếm số lượng phần tử trong mảng A có giá trị lớn hơn avg .
- Kiểm tra xem chuỗi S có phải là chuỗi đối xứng (Palindrome) hay không (chuỗi đọc xuôi hay đọc ngược đều giống nhau).

Đầu vào (Input):

- Dòng 1: Số nguyên N .
- Dòng 2: N số nguyên của mảng A .
- Dòng 3: Chuỗi ký tự S .

Đầu ra (Output): Số lượng phần tử lớn hơn trung bình cộng và nhân YES (nếu đối xứng) hoặc NO (nếu không đối xứng) cách nhau dấu cách.

Ví dụ mẫu:

Đầu vào (Input):

3
1 2 3
aba

Đầu ra (Output):

1 YES

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì):

Đáp án đối chiếu xem tại Phụ lục cuối sách

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	3 1 2 3 aba	
2	Warm-up	4 2 4 6 8 noon	
3	Warm-up	3 5 5 5 cat	
4	Edge case	1 10 z	
5	Edge case	2 1 9 no	
6	Edge case	2 4 4 gg	
7	Trick case	5 -10 0 10 20 30 racecar	
8	Trick case	4 -5 -5 -5 -1 abba	
9	Mental challenge	5 1 3 5 7 9 radar	
10	Mental challenge	6 2 2 2 8 8 8 level	

Chương 5: Địa Chỉ Ô Nhớ, Bản Chất Con Trỏ & Toán Tử &, *

[THUẬT NGỮ KHOA HỌC] Con Trỏ (Pointer)

Từ nguyên học: Được nhà khoa học máy tính Harold Lawson phát minh năm 1964 trong ngôn ngữ PL/I, đặt nền tảng cho việc quản lý bộ nhớ trực tiếp trong các ngôn ngữ hệ thống như C và C++.

Mô hình trực giác: Hãy hình dung mỗi ngôi nhà trên phố có một 'Số nhà' (Địa chỉ ô nhớ) và bên trong chứa 'Đồ đạc' (Giá trị dữ liệu). Con trỏ không phải là ngôi nhà; con trỏ là một mẫu giấy ghi lại con số 'Số nhà' đó. Khi cầm mẫu giấy, bạn có thể đi đến ngôi nhà tương ứng để xem hoặc sửa đổi đồ đạc bên trong.

5.1 Toán Tử Lấy Địa Chỉ Ô Nhớ & (Address-Of)

Mọi biến số khai báo trong chương trình đều ngự trị tại một địa chỉ cụ thể trên dải ô nhớ RAM. Toán tử đơn phân & đặt trước tên biến cho phép chúng ta lấy ra tọa độ ô nhớ vật lý này:

```
int x = 42;  
// &x là địa chỉ của ô nhớ chứa giá trị 42 (ví dụ: 0x7ffd58)
```

5.2 Bản Chất Con Trỏ & Toán Tử Giải Tham Chiếu * (Dereferencing)

Con trỏ đơn thuần là một biến số có kích thước 8 Bytes (trên hệ điều hành 64-bit), có nhiệm vụ duy nhất là lưu trữ địa chỉ của một biến khác:

[MÔ XẺ CÚ PHÁP C++14]	
<pre>int* ptr = &x; *ptr = 100;</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<pre>int*</pre>	Khai báo kiểu dữ liệu: `ptr` là một con trỏ có khả năng trỏ tới ô nhớ kiểu `int`.
<pre>= &x;</pre>	Nạp địa chỉ của biến `x` vào ô nhớ của con trỏ `ptr`.
<pre>*ptr</pre>	Toán tử giải tham chiếu (dereference): Đi theo địa chỉ đang lưu trong `ptr` để tiếp cận ô nhớ gốc `x`.
<pre>= 100;</pre>	Ghi đè giá trị 100 thẳng vào ô nhớ `x`. Lúc này giá trị của `x` biến thành 100!

5.3 Số Học Con Trỏ & Đẳng Thức Nền Tảng $a[i] == *(a + i)$

Phép toán cộng trừ trên con trỏ không phải là cộng trừ byte thông thường, mà là nhảy theo **bước nhảy kích thước kiểu dữ liệu** ($\text{sizeof}(T)$):

- Nếu `ptr` trỏ tới `int` tại địa chỉ `1000`, thì `ptr + 1` sẽ có địa chỉ `1004` (vì kiểu `int` chiếm 4 bytes).
- Nếu `ptr` trỏ tới `double` tại địa chỉ `1000`, thì `ptr + 1` sẽ có địa chỉ `1008`.

[LÝ THUYẾT] Bí mật đằng sau toán tử ngoặc vuông mảng []

Trong ngôn ngữ C/C++, tên của một mảng tĩnh `a` tự động suy biến thành con trỏ hằng trỏ vào phần tử đầu tiên:

$$a \equiv a[0]$$

Do đó, cú pháp truy xuất phần tử `a[i]` thực chất chỉ là “đường tắt cú pháp” (syntactic sugar) cho phép giải tham chiếu sau khi dời con trỏ đi i bước:

$$a[i] \equiv *(a + i)$$

5.4 Chạy Bàn Mô Hình Con Trỏ Trên Giấy

Cho mảng `int a[3] = {10, 20, 30}` và con trỏ `ptr = a`:

Biểu thức	Địa chỉ ô nhớ giả định	Giá trị đọc được	Ý nghĩa thao tác
<code>ptr</code>	0x100	0x100	Con trỏ giữ địa chỉ phần tử đầu tiên
<code>*ptr</code>	0x100	10	Giải tham chiếu <code>a[0]</code>
<code>ptr + 1</code>	0x104	0x104	Dời con trỏ sang phần tử tiếp theo (+4 bytes)
<code>*(ptr + 1)</code>	0x104	20	Giải tham chiếu <code>a[1]</code>
<code>*(ptr + 2)</code>	0x108	30	Giải tham chiếu <code>a[2]</code>

5.5 Bài Tập Tiêu Chuẩn

Bài toán: Bài 5.1 - Tìm Cực Trị Bằng Con Trỏ & Khoảng Cách Ô Nhớ (Pointer MinMax)

(Nguồn: Nền tảng C++14 - Thao tác Con trỏ thuần túy)

Cho một mảng số nguyên A gồm N phần tử ($1 \leq N \leq 8$).

Sử dụng thuần túy con trỏ `int* ptr = a;` và phép số học con trỏ `*(ptr + i)` (không dùng cú pháp dấu ngoặc vuông `[]`), em hãy tìm:

1. Giá trị nhỏ nhất (`min_val`) và vị trí xuất hiện đầu tiên của nó (`min_idx`).

2. Giá trị lớn nhất (`max_val`) và vị trí xuất hiện đầu tiên của nó (`max_idx`).

3. Tính khoảng cách vị trí ô nhỏ giữa phần tử Max và Min: `offset = max_idx - min_idx`.

Đầu vào (Input): Dòng 1 chứa *N*. Dòng 2 chứa *N* số nguyên.

Đầu ra (Output): In ra 3 số nguyên `min_val max_val offset` cách nhau một khoảng trắng.

Ví dụ mẫu:

Đầu vào (Input):

3

10 20 30

Đầu ra (Output):

10 30 2

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì):

Đáp án đối chiếu xem tại Phụ lục cuối sách

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	3 10 20 30	[.....]
2	Warm-up	3 30 20 10	[.....]
3	Warm-up	4 5 1 9 3	[.....]
4	Edge case	1 42	[.....]
5	Edge case	2 7 7	[.....]
6	Edge case	2 -10 10	[.....]
7	Trick case	5 -8 -3 -15 0 -2	[.....]
8	Trick case	4 4 2 2 4	[.....]
9	Mental challenge	6 12 5 28 3 19 28	[.....]
10	Mental challenge	5 -5 -2 -9 -1 -4	[.....]

Chương 6: Hàm, Tham Chiếu, Tham Trị & Ngăn Xếp Gọi Hàm (Call Stack)

[THUẬT NGỮ KHOA HỌC] Ngăn Xếp Gọi Hàm (Call Stack)

Từ nguyên học: Được nhà khoa học máy tính người Đức Klaus Samelson và Friedrich L. Bauer đề xuất năm 1957 trong việc biên dịch biểu thức số học.

Mô hình trực giác: Một chồng đĩa trong quán ăn. Mỗi khi bạn gọi một hàm mới, bạn đặt thêm một chiếc đĩa (Stack Frame) lên đỉnh chồng đĩa. Mọi biến cục bộ của hàm nằm gọn trong chiếc đĩa đó. Khi hàm hoàn thành công việc và trả về kết quả, chiếc đĩa được nhấc ra khỏi đỉnh để quay lại chiếc đĩa bên dưới.

6.1 Phân Biệt Triệt Đê: Tham Trị (Pass-by-Value) & Tham Chiếu (Pass-by-Reference)

Trong C++, cách chúng ta truyền biến vào hàm quyết định việc ô nhớ mới có được tạo ra hay không:

Phương thức	Cú pháp hàm	Bản chất ô nhớ
Truyền Giá Trị (Pass-by-Value)	<code>void func(int x)</code>	Hệ thống sao chép giá trị sang một ô nhớ hoàn toàn mới trong Stack Frame của <code>func</code> . Mọi thay đổi đối với <code>x</code> không hề ảnh hưởng đến biến gốc ở hàm gọi.
Truyền Tham Chiếu (Pass-by-Reference)	<code>void func(int& x)</code>	Dấu <code>&</code> ở đây KHÔNG PHẢI toán tử lấy địa chỉ, mà là cú pháp khai báo Bí danh (Alias) . Biến <code>x</code> bên trong hàm dùng chung địa chỉ với biến gốc. Mọi thay đổi đối với <code>x</code> sẽ làm thay đổi trực tiếp biến gốc!

[MỖ XÉ CÚ PHÁP C++14]

```
void swap(int& a, int& b) { int temp = a; a = b; b = temp; }
```

Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>int& a, int& b</code>	Nhận bí danh tham chiếu của 2 biến truyền vào. Không tạo ô nhớ sao chép mới.
<code>int temp = a;</code>	Lưu giá trị của biến gốc thứ nhất vào biến tạm thời cục bộ <code>temp</code> .
<code>a = b;</code>	Ghi đè giá trị biến gốc thứ hai vào biến gốc thứ nhất.
<code>b = temp;</code>	Hoàn tất hoán đổi hai biến gốc bên ngoài phạm vi hàm!

6.2 Mô Hình Khung Ngăn Xếp (Stack Frame) & Độ Quy Euclid

Khi một hàm đệ quy tự gọi chính nó, mỗi lần gọi sẽ tạo ra một **Khung ngăn xếp (Stack Frame)** riêng biệt xếp chồng lên nhau.

Giả sử ta gọi hàm tính ước chung lớn nhất $\text{gcd}(12, 8)$:

- Bước 1 (Đáy Stack):** $\text{gcd}(12, 8)$ với $\text{depth} = 1$. Vì $b = 8 \neq 0$, gọi tiếp $\text{gcd}(8, 4)$.
- Bước 2 (Chồng thêm):** $\text{gcd}(8, 4)$ với $\text{depth} = 2$. Vì $b = 4 \neq 0$, gọi tiếp $\text{gcd}(4, 0)$.
- Bước 3 (Đỉnh Stack):** $\text{gcd}(4, 0)$ với $\text{depth} = 3$. Chạm điều kiện dừng (Base case) $b = 0$, trả về $a = 4$.
- Bước 4 (Giải phóng):** Lần lượt các khung ngăn xếp được pop ra khỏi đỉnh và trả giá trị về $\text{main}()$.

[LƯU Ý QUAN TRỌNG] Nguy cơ Tràn Ngăn Xếp (Stack Overflow)

Bộ nhớ dành cho Call Stack là hữu hạn (thường khoảng 8MB trên Linux hoặc máy chấm thi). Nếu bạn viết hàm đệ quy thiếu điều kiện dừng hoặc đệ quy quá sâu ($\text{depth} > 10^5$), Call Stack sẽ tràn qua vùng nhớ khác và chương trình bị hệ điều hành tiêu diệt ngay lập tức với mã lỗi **RTE (Runtime Error / Segmentation fault)**!

6.3 Chạy Bàn Khung Ngăn Xếp Trên Giấy

Theo dõi trạng thái đỉnh Stack khi tìm $\text{gcd}(12, 8)$:

Bước	Khung ngăn xếp đỉnh	Tham số a	Tham số b	Độ sâu depth	Hành động
1	$\text{gcd}(12, 8)$	12	8	1	Push khung 1, gọi tiếp
2	$\text{gcd}(8, 4)$	8	4	2	Push khung 2, gọi tiếp
3	$\text{gcd}(4, 0)$	4	0	3	Push khung 3, chạm Base Case!
4	Pop khung 3	4	0	3	Return 4 về khung 2
5	Pop khung 2	8	4	2	Return 4 về khung 1
6	Pop khung 1	12	8	1	Return 4 về hàm $\text{main}()$

6.4 Bài Tập Tiêu Chuẩn

Bài toán: Bài 6.1 - Khung Ngăn Xếp Đệ Quy & Ước Số Chung Lớn Nhất (Call Stack & GCD)

(Nguồn: Nền tảng C++14 - Tham chiếu & Call Stack)

Cài đặt hàm tìm ước chung lớn nhất Euclid theo mô hình đệ quy:

```
int gcd(int a, int b, int depth, int& max_depth)
```

- Tham số `depth`: truyền theo tham trị để ghi nhận độ sâu của khung ngăn xếp hiện tại (bắt đầu từ 1).
- Tham số `max_depth`: truyền theo tham chiếu `int&` để cập nhật độ sâu lớn nhất mà Call Stack đạt được trong toàn bộ chu kỳ sống.
- Tại mỗi khung, nếu `depth > max_depth`, gán `max_depth = depth`.
- Nếu $b = 0$, trả về a ; ngược lại gọi đệ quy `gcd(b, a % b, depth + 1, max_depth)`.

Đầu vào (Input): Hai số nguyên A, B ($1 \leq A, B \leq 100$).

Đầu ra (Output): Hai số nguyên `gcd_result max_depth` cách nhau dấu cách.

Ví dụ mẫu:

Đầu vào (Input):

12 8

Đầu ra (Output):

4 3

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	12 8	[.....]
2	Warm-up	10 5	[.....]
3	Warm-up	7 7	[.....]
4	Edge case	13 0	[.....]
5	Edge case	1 1	[.....]
6	Edge case	1 25	[.....]
7	Trick case	18 12	[.....]
8	Trick case	21 14	[.....]
9	Mental challenge	34 21	[.....]
10	Mental challenge	48 18	[.....]

Chương 7: Struct & Mỏ Xẻ Cú Pháp operator<

[THUẬT NGỮ KHOA HỌC] Cấu Trúc Tự Định Nghĩa (Struct)

Từ nguyên học: Xuất phát từ từ 'Structure' trong tiếng Anh, được đưa vào ngôn ngữ C từ năm 1972 để biểu diễn các bản ghi dữ liệu phức hợp gồm nhiều trường thông tin.

Mô hình trực giác: Một 'chiếc túi' gom nhóm nhiều món đồ có kiểu khác nhau (như số nguyên, chuỗi ký tự, số thực) lại thành một thực thể duy nhất có ý nghĩa trọn vẹn trong bài toán (ví dụ: một Cảnh đồ thị, một Tọa độ điểm, một Hồ sơ thí sinh).

7.1 Đóng Gói Dữ Liệu Với `struct`

Trong lập trình thi đấu, thay vì quản lý nhiều mảng song song rời rạc (`x[i]`, `y[i]`, `id[i]`), ta gom chúng vào một `struct`:

```
struct Point {  
    int id;  
    int x;  
    int y;  
};
```

Khi khai báo `Point p;`, bộ nhớ RAM sẽ cấp phát liên tiếp 3 biến `id`, `x`, `y` kề nhau ($3 \times 4 = 12$ bytes).

7.2 Mỏ Xẻ Tận Gốc Cú Pháp `bool operator<(const Edge& other) const`

Khi chúng ta muốn dùng hàm sắp xếp chuẩn `std::sort` trên một mảng các `struct`, trình biên dịch cần biết làm thế nào để so sánh giữa hai đối tượng. Ta nạp chồng toán tử `<`:

[MỔ XẺ CÚ PHÁP C++14]	
<pre>bool operator<(const Edge& other) const { return weight < other.weight; }</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<code>bool</code>	Kiểu giá trị trả về: `true` nếu đối tượng hiện tại đứng trước đối tượng `other`, `false` nếu ngược lại.
<code>operator<</code>	Tên hàm đặc biệt trong C++ tái định nghĩa hành vi so sánh nhỏ hơn.
<code>const Edge&</code>	1. `Edge&`: Nhận địa chỉ tham chiếu tránh sao chép toàn bộ struct gây lãng phí thời gian và RAM. 2. `const` đầu: Khóa đối tượng `other` ở chế độ chỉ đọc (Read-only), ngăn ngừa mọi hành vi vô tình sửa đổi dữ liệu của `other`.
<code>const</code> đuôi	Khóa đối tượng gọi hàm (`*this`) ở chế độ chỉ đọc. ĐÂY LÀ ĐIỀU KIỆN BẮT BUỘC trong C++: các thuật toán STL như `std::sort`, `std::priority_queue`, `std::set` đều truyền phần tử dưới dạng hằng số (`const`). Nếu thiếu `const` ở đuôi, trình biên dịch sẽ từ chối biên dịch ngay lập tức!

7.3 Quy Trình So Sánh Đa Tiêu Chí Trên Giấy

- Khi sắp xếp tọa độ điểm theo 3 tiêu chí:
- Tiêu chí 1: Điểm có khoảng cách Manhattan $D = |x| + |y|$ nhỏ hơn đứng trước.
 - Tiêu chí 2: Nếu D bằng nhau, điểm có hoành độ x nhỏ hơn đứng trước.
 - Tiêu chí 3: Nếu cả D và x bằng nhau, điểm có mã số `id` nhỏ hơn đứng trước.
- Bảng so sánh tay hai điểm P_1 ($id = 1, x = 2, y = 3$) và P_2 ($id = 2, x = 1, y = 1$):

Đối tượng	Manhattan $D = x + y $	Hoành độ x	Mã id	Kết quả so sánh
P1	<code>2 + 3 = 5</code>	2	1	<code>D1 = 5</code>
P2	<code>1 + 1 = 2</code>	1	2	<code>D2 = 2</code>
Kết luận	-	-	-	<code>D2 < D1 ⇒ P2 đứng trước P1</code>

7.4 Bài Tập Tiêu Chuẩn

Bài toán: Bài 7.1 - Sắp Xếp Tọa Độ Điểm (Point Struct Sort) (Nguồn: Nền tảng C++14 - Struct & Nạp chồng operator<)
Cho N điểm trên mặt phẳng tọa độ ($1 \leq N \leq 8$). Mỗi điểm gồm mã số <code>id</code> , hoành độ <code>x</code> và tung độ <code>y</code> .

Em hãy cài đặt `struct Point` với phương thức so sánh `bool operator<(const Point& other) const` theo thứ tự ưu tiên:

- Khoảng cách Manhattan $D = |x| + |y|$ nhỏ hơn đứng trước.
- Nếu khoảng cách bằng nhau, hoành độ `x` nhỏ hơn đứng trước.
- Nếu hoành độ vẫn bằng nhau, `id` nhỏ hơn đứng trước.

Đầu vào (Input): Dòng 1 chứa số nguyên N . N dòng tiếp theo, mỗi dòng chứa 3 số nguyên `id x y`.

Đầu ra (Output): Dãy mã số `id` của các điểm sau khi sắp xếp trên một dòng cách nhau dấu cách.

Ví dụ mẫu:

Đầu vào (Input):

2
1 2 3
2 1 1

Đầu ra (Output):

2 1

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	2 1 2 3 2 1 1	
2	Warm-up	3 1 0 5 2 1 2 3 0 1	
3	Warm-up	2 1 3 0 2 0 3	
4	Edge case	1 99 0 0	
5	Edge case	2 1 0 0 2 0 0	
6	Edge case	3 1 -2 0 2 2 0 3 0 2	
7	Trick case	4 1 1 1 2 -1 -1 3 -1 1 4 1 -1	
8	Trick case	3 10 0 4 20 2 2 30 -4 0	

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
9	Mental challenge	5 1 3 3 2 1 1 3 2 2 4 0 2 5 1 0	[.....]
10	Mental challenge	4 4 5 5 3 -2 -3 2 0 1 1 -1 0	[.....]

Chương 8: Phân Tích Độ Phức Tạp Thuật Toán Trên Giấy

[THUẬT NGỮ KHOA HỌC]

Ký Hiệu Big-O (Asymptotic Notation)

Từ nguyên học:

Được nhà toán học người Đức Paul Bachmann giới thiệu năm 1894 và được Donald Knuth phổ biến trong khoa học máy tính vào năm 1976.

Mô hình trực giác:

Một 'chiếc thước đo tốc độ' phản ánh xu hướng bùng nổ thời gian thực thi của thuật toán khi kích thước dữ liệu N tăng lên vô tận. Big-O bỏ qua các hệ số nhân và hằng số nhỏ để tập trung vào bậc tăng trưởng cao nhất.

8.1 Định Nghĩa Toán Học Về Tiệm Cận Chặn Trên

Hàm số $f(N)$ được gọi là $O(g(N))$ nếu tồn tại hai hằng số dương $c > 0$ và $N_0 > 0$ sao cho:

$$f(N) \leq c \times g(N) \quad \forall N \geq N_0$$

Trong lập trình thi đấu, ta luôn phân tích trường hợp xấu nhất (Worst-case time complexity) để đảm bảo thuật toán không bị quá thời gian (TLE - Time Limit Exceeded) trước bất kỳ bộ test hiểm hóc nào của ban giám khảo.

8.2 Quy Tắc Ngón Tay Cái 1 $\text{text}\{s\} \approx 10^8$ Phép Tính

Hầu hết các hệ thống chấm thi tự động (Codeforces, VNOJ, ICPC, HSG Quốc Gia) đều sử dụng CPU có tốc độ thực hiện khoảng 10^8 phép tính cơ bản trong 1 giây:

Giới hạn N đề bài	Độ phức tạp mục tiêu	Thuật toán phù hợp
$N \leq 10$	$O(N!)$	Duyệt toàn bộ hoán vị, quay lui đầy đủ
$N \leq 20$	$O(2^N)$	Duyệt tập con, quay lui nhị phân, DP bitmask
$N \leq 500$	$O(N^3)$	Floyd-Warshall, nhân ma trận
$N \leq 5,000$	$O(N^2)$	Hai vòng lặp lồng nhau, quy hoạch động 2 chiều cơ bản
$N \leq 10^5 \dots 10^6$	$O(N \log N)$ hoặc $O(N \sqrt{N})$	Sắp xếp, Tìm kiếm nhị phân, Hai con trỏ, Cây phân đoạn
$N \leq 10^{18}$	$O(\log N)$ hoặc $O(1)$	Lũy thừa nhị phân, Euclid GCD, Công thức toán học

8.3 Kỹ Thuật Đếm Thao Tác Hai Vòng Lặp Lồng Nhau

Xét đoạn mã kiểm tra toàn bộ các cặp chỉ số phân biệt (i, j) với $0 \leq i < j < N$:

```
int ops = 0;
for (int i = 0; i < N; ++i) {
    for (int j = i + 1; j < N; ++j) {
        ops++;
    }
}
```

Tổng số lần thực thi thân vòng lặp trong là:

$$T(N) = \sum_{i=0}^{N-1} (N-1-i) = (N-1) + (N-2) + \dots + 1 + 0 = \frac{N(N-1)}{2} = \frac{N^2}{2} - \frac{N}{2} \in O(N^2)$$

[CHẠY BÀN] Ước lượng thời gian chạy trên giấy khi mở rộng $N = 100,000$

Nếu đề bài cho $N = 100,000 = 10^5$:

$$T(10^5) = \frac{10^5 \times (10^5 - 1)}{2} \approx 5 \times 10^9 \text{ phép tính}$$

Với tốc độ 10^8 phép tính/giây của CPU:

$$\text{Thời gian thực thi} \approx \frac{5 \times 10^9}{10^8} = 50 \text{ giây!}$$

Thời gian này vượt xa giới hạn 1.0 giây của máy chấm, chương trình sẽ lập tức bị TLE. Phân tích này cho ta biết ngay trên giấy rằng thuật toán $O(N^2)$ không đạt yêu cầu mà phải tìm thuật toán $O(N \log N)$!

8.4 Bài Tập Tiêu Chuẩn

Bài toán: Bài 8.1 - Đếm Thao Tác Vòng Lặp & Đánh Giá Giới Hạn (Loop Operation Counter) (Nguồn: Nền tảng C++14 - Phân tích Big-O & Ước lượng 10^8)

Cho mảng số nguyên A gồm N phần tử ($1 \leq N \leq 8$).

Xét đoạn thuật toán duyệt hai vòng lặp lồng nhau:

- Với mỗi cặp (i, j) thỏa mãn $0 \leq i < j < N$: tăng biến đếm thao tác `ops`. Nếu $A[i] > A[j]$, tăng biến đếm số lượng cặp nghịch thế `inv`.
- Dựa vào kết quả lý thuyết, với $N' = 10^5$, thuật toán $O(N^2)$ này sẽ mất xấp xỉ bao nhiêu giây theo quy tắc 10^8 phép tính/giây? (Quy ước hằng số lý thuyết làm tròn là 50 giây).

Đầu vào (Input): Dòng 1 chứa số nguyên N . Dòng 2 chứa N số nguyên.

Đầu ra (Output): In ra 3 số nguyên `ops inv 50` cách nhau dấu cách.

Ví dụ mẫu:

Đầu vào (Input):

```
3
3 2 1
```

Đầu ra (Output):

```
3 3 50
```

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): Đáp án đối chiếu xem tại Phụ lục cuối sách

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	2 1 2	[.....]
2	Warm-up	2 2 1	[.....]
3	Warm-up	3 1 2 3	[.....]
4	Edge case	1 5	[.....]
5	Edge case	3 3 2 1	[.....]
6	Edge case	3 2 2 2	[.....]
7	Trick case	4 4 3 2 1	[.....]
8	Trick case	4 1 3 2 4	[.....]
9	Mental challenge	5 2 4 1 3 5	[.....]
10	Mental challenge	5 5 1 2 3 4	[.....]

Bài Tập Tích Hợp Tập 1: Quản Lý Hồ Sơ Thí Sinh

[THUẬT NGỮ KHOA HỌC] Sắp Xếp Con Trỏ (Pointer-based Sorting)

Từ nguyên học: Kỹ thuật tối ưu hóa quản lý bộ nhớ kinh điển trong lập trình hệ thống C/C++.

Mô hình trực giác: Thay vì di chuyển những chiếc tủ nặng nề (sao chép toàn bộ khối dữ liệu struct lớn), ta chỉ sắp xếp lại những chiếc nhãn dán ghi số phòng (hoán đổi con trỏ địa chỉ 8-byte). Dữ liệu gốc vẫn nằm yên tại chỗ trên thanh RAM, nhưng thứ tự truy xuất đã được sắp xếp hoàn hảo.

1. Bối Cảnh Thực Tế & Kiến Trúc Tích Hợp

Một hệ thống quản lý kỳ thi Olympic cần xử lý danh sách N thí sinh ($1 \leq N \leq 8$). Mỗi thí sinh được định nghĩa bởi một `struct Student`:

```
struct Student {  
    int id;      // Mã số báo danh  
    int score;   // Điểm số bài thi  
    int age;     // Tuổi của thí sinh  
};
```

Hệ thống tích hợp toàn bộ các kỹ thuật đã học trong Tập 1:

- Mảng Tĩnh (Chương 4):** Dữ liệu được lưu trong mảng `Student arr[16]`.
- Con Trỏ & Địa Chỉ (Chương 5):** Tạo mảng con trỏ `Student* ptrs[16]` và gán `ptrs[i] = &arr[i]`.
- So Sánh Đa Tiêu Chí (Chương 7):**
 - Ưu tiên 1: Thí sinh có điểm số (`score`) cao hơn xếp trước.
 - Ưu tiên 2: Nếu bằng điểm, thí sinh ít tuổi hơn (`age` nhỏ hơn) xếp trước.
 - Ưu tiên 3: Nếu bằng cả điểm lẫn tuổi, thí sinh có mã số (`id`) nhỏ hơn xếp trước.
- Thuật Toán Chọn Trục Tiếp & Phân Tích Độ Phức Tạp (Chương 8):**
 - Sắp xếp mảng con trỏ bằng thuật toán Selection Sort.
 - Tổng số phép so sánh thực hiện giữa các con trỏ luôn bằng:

$$C(N) = \frac{N(N-1)}{2} \in O(N^2)$$

[MỖ XẼ CÚ PHÁP C++14]	
<pre>Student* temp = ptrs[i]; ptrs[i] = ptrs[best]; ptrs[best] = temp;</pre>	
Thành phần cú pháp	Cơ chế ngữ nghĩa & Mục đích
<pre>Student* temp = ptrs[i];</pre>	Lưu trữ con trỏ thứ `i` (chỉ chiếm 8 bytes trên RAM) vào con trỏ tạm.
<pre>ptrs[i] = ptrs[best];</pre>	Trò con trỏ vị trí `i` sang đối tượng tối ưu nhất.
<pre>ptrs[best] = temp;</pre>	Hoàn tất hoán đổi 2 con trỏ mà KHÔNG CẦN copy bất kỳ byte dữ liệu nào của `Student`!

2. Bảng Theo Dõi Chạy Bàn Trên Giấy

- Với $N = 3$ thí sinh:
- Thí sinh 1 (S_1): `id = 1, score = 70, age = 16`
 - Thí sinh 2 (S_2): `id = 2, score = 95, age = 15`
 - Thí sinh 3 (S_3): `id = 3, score = 85, age = 17`

Vòng lặp i	Phép so sánh thực hiện	Con trỏ tốt nhất (best)	Mảng con trỏ sau bước i
i = 0	(ptrs[1], ptrs[0]): 95 > 70 ⇒ best=1 (ptrs[2], ptrs[1]): 85 < 95	best = 1 (id=2)	[id=2, id=1, id=3]
i = 1	(ptrs[2], ptrs[1]): 85 > 70 ⇒ best=2	best = 2 (id=3)	[id=2, id=3, id=1]
Tổng kết	Tổng số phép so sánh: 2 + 1 = 3	Top: id=2 (95đ)	[2, 3, 1]

3. Bài Tập Tích Hợp Tiêu Chuẩn

Bài toán: Bài Tập Tích Hợp 1 - Quản Lý Hồ Sơ Thí Sinh (Student Records) (Nguồn: Chuyên khảo CP C++14 - Tích hợp Toàn diện Tập 1)
Cho danh sách N thí sinh ($1 \leq N \leq 8$). Mỗi thí sinh gồm 3 thông tin <code>id score age</code>. Em hãy thực hiện: - Đọc dữ liệu vào mảng <code>Student arr[N]</code> và khởi tạo mảng con trỏ <code>Student* ptrs[N]</code>. - Sắp xếp mảng con trỏ theo tiêu chí: Điểm cao xếp trước; cùng điểm thì ít tuổi hơn xếp trước; cùng tuổi thì <code>id</code> nhỏ hơn xếp trước. Đếm tổng số phép so sánh cặp thí sinh đã thực hiện trong thuật toán Selection Sort.

3. In ra danh sách `id` sau khi sắp xếp trên dòng 1. Dòng 2 in: `top_id top_score total_comparisons`.

Đầu vào (Input): Dòng 1 chứa N . N dòng tiếp theo, mỗi dòng chứa 3 số nguyên `id score age`.

Đầu ra (Output): Dòng 1 gồm N mã số `id`. Dòng 2 gồm 3 số nguyên `top_id top_score total_comparisons`.

Ví dụ mẫu:

Đầu vào (Input):

```
3
1 70 16
2 95 15
3 85 17
```

Đầu ra (Output):

```
2 3 1
2 95 3
```

Thử thách 10 Test Bàn Tay (Tính nhẩm & Điền bằng bút chì): *Đáp án đối chiếu xem tại Phụ lục cuối sách*

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
1	Warm-up	2 1 90 15 2 80 16	[.....]
2	Warm-up	3 1 70 16 2 95 15 3 85 17	[.....]
3	Warm-up	2 1 85 16 2 85 15	[.....]
4	Edge case	1 42 100 18	[.....]
5	Edge case	2 1 80 15 2 80 15	[.....]
6	Edge case	3 10 50 14 20 50 14 30 50 14	[.....]
7	Trick case	4 1 90 17 2 90 16 3 95 18 4 95 15	[.....]
8	Trick case	3 1 60 19 2 70 18 3 60 17	[.....]

Test	Phân loại	Dữ liệu vào (Input)	Đáp án của bạn (Bút chì)
9	Mental challenge	5 1 50 15 2 80 16 3 65 14 4 90 17 5 80 15	[.....]
10	Mental challenge	4 10 100 20 20 100 18 30 100 19 40 100 18	[.....]

Phụ Lục: Đáp Án & Bảng Vết Thực Thi 10 Test Bàn Tay

[LÝ THUYẾT] Hướng Dẫn Đối Chiếu Lời Giải Bằng Bút Chì

Phụ lục này cung cấp toàn bộ đáp án chính thức và bảng vết thực thi (Execution Trace) từng bước cho tất cả 10 bài test của từng chương. Sau khi đã tự giải và điền kết quả vào các ô trống bằng bút chì, các em hãy đối chiếu với bảng đáp án dưới đây để phát hiện chính xác bước tư duy nào bị nhầm lẫn.

Lời Giải Chi Tiết Chương 1: Hoán Đổi Ô Nhớ RAM

Lời giải & Đáp án: Bài 1.1 - Hoán Đổi Ô Nhớ RAM			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	3 5	5 3 8	Hai số nguyên dương nhỏ cơ bản
2	10 20	20 10 30	Hai số tròn chục cơ bản
3	1 1	1 1 2	Hai số bằng nhau
4	0 0	0 0 0	Trường hợp hai số đều bằng 0
5	0 7	7 0 7	Một số bằng 0
6	-5 5	5 -5 0	Hai số đối dấu có tổng bằng 0
7	-10 -20	-20 -10 -30	Hai số nguyên âm
8	-8 0	0 -8 -8	Số âm và số 0
9	42 17	17 42 59	Tính nhầm hai số khác biệt
10	-15 35	35 -15 20	Tính nhầm số âm và số dương

Bảng Vết Thực Thi Từng Bước (Chương 1)

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	3
-	b	5
2	Calculate sum = a + b	-
-	sum	8
3	Assign temp = a	-
-	temp	3
4	Assign a = b	-
-	a	5
5	Assign b = temp	-
-	b	3

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	10
-	b	20
2	Calculate sum = a + b	-
-	sum	30
3	Assign temp = a	-
-	temp	10
4	Assign a = b	-
-	a	20
5	Assign b = temp	-
-	b	10

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	1
-	b	1
2	Calculate sum = a + b	-
-	sum	2
3	Assign temp = a	-
-	temp	1
4	Assign a = b	-
-	a	1
5	Assign b = temp	-
-	b	1

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	0
-	b	0
2	Calculate sum = a + b	-
-	sum	0
3	Assign temp = a	-
-	temp	0
4	Assign a = b	-
-	a	0
5	Assign b = temp	-
-	b	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	0
-	b	7

Bước	Mô tả / Biến	Giá trị
2	Calculate sum = a + b	-
-	sum	7
3	Assign temp = a	-
-	temp	0
4	Assign a = b	-
-	a	7
5	Assign b = temp	-
-	b	0

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	-5
-	b	5
2	Calculate sum = a + b	-
-	sum	0
3	Assign temp = a	-
-	temp	-5
4	Assign a = b	-
-	a	5
5	Assign b = temp	-
-	b	-5

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	-10
-	b	-20
2	Calculate sum = a + b	-
-	sum	-30
3	Assign temp = a	-
-	temp	-10
4	Assign a = b	-
-	a	-20
5	Assign b = temp	-
-	b	-10

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	-8
-	b	0
2	Calculate sum = a + b	-
-	sum	-8
3	Assign temp = a	-

Bước	Mô tả / Biến	Giá trị
-	temp	-8
4	Assign a = b	-
-	a	0
5	Assign b = temp	-
-	b	-8

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	42
-	b	17
2	Calculate sum = a + b	-
-	sum	59
3	Assign temp = a	-
-	temp	42
4	Assign a = b	-
-	a	17
5	Assign b = temp	-
-	b	42

Bước	Mô tả / Biến	Giá trị
1	Read inputs a and b	-
-	a	-15
-	b	35
2	Calculate sum = a + b	-
-	sum	20
3	Assign temp = a	-
-	temp	-15
4	Assign a = b	-
-	a	35
5	Assign b = temp	-
-	b	-15

Lời Giải Chi Tiết Chương 2: Tích An Toàn & Ngưỡng Giới Hạn

Lời giải & Đáp án: Bài 2.1 - Tích An Toàn & Ngưỡng Giới Hạn			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	3 4 50	SAFE 12	Tích 12 nằm an toàn dưới ngưỡng 50
2	5 6 20	OVERFLOW 30	Tích 30 vượt ngưỡng 20
3	2 10 20	SAFE 20	Tích 20 đúng bằng ngưỡng 20
4	0 100 50	SAFE 0	Nhân với 0 luôn bằng 0
5	10 0 0	SAFE 0	Ngưỡng 0 và tích bằng 0
6	-5 6 35	SAFE -30	Tích âm có giá trị tuyệt đối 30 dưới ngưỡng 35
7	-5 6 25	OVERFLOW -30	Tích âm có giá trị tuyệt đối 30 vượt ngưỡng 25
8	-4 -7 25	OVERFLOW 28	Hai số âm nhân nhau ra dương 28 vượt ngưỡng 25
9	12 8 100	SAFE 96	Tính nhẩm $12 \times 8 = 96 \leq 100$
10	-15 7 100	OVERFLOW -105	Tính nhẩm $ -105 > 100$

Bảng Vết Thực Thi Từng Bước (Chương 2)

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	3
-	b	4
-	threshold	50
2	Compute <code>static_cast<long long>(a) * b</code>	-
-	prod	12
-	abs_prod	12
3	abs_prod within threshold: SAFE	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	5
-	b	6
-	threshold	20
2	Compute <code>static_cast<long long>(a) * b</code>	-
-	prod	30
-	abs_prod	30
3	abs_prod exceeds threshold: OVERFLOW	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	2

Bước	Mô tả / Biến	Giá trị
-	b	10
-	threshold	20
2	Compute static_cast<long long>(a) * b	-
-	prod	20
-	abs_prod	20
3	abs_prod within threshold: SAFE	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	0
-	b	100
-	threshold	50
2	Compute static_cast<long long>(a) * b	-
-	prod	0
-	abs_prod	0
3	abs_prod within threshold: SAFE	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	10
-	b	0
-	threshold	0
2	Compute static_cast<long long>(a) * b	-
-	prod	0
-	abs_prod	0
3	abs_prod within threshold: SAFE	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	-5
-	b	6
-	threshold	35
2	Compute static_cast<long long>(a) * b	-
-	prod	-30
-	abs_prod	30
3	abs_prod within threshold: SAFE	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	-5
-	b	6
-	threshold	25

Bước	Mô tả / Biến	Giá trị
2	Compute static_cast<long long>(a) * b	-
-	prod	-30
-	abs_prod	30
3	abs_prod exceeds threshold: OVERFLOW	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	-4
-	b	-7
-	threshold	25
2	Compute static_cast<long long>(a) * b	-
-	prod	28
-	abs_prod	28
3	abs_prod exceeds threshold: OVERFLOW	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	12
-	b	8
-	threshold	100
2	Compute static_cast<long long>(a) * b	-
-	prod	96
-	abs_prod	96
3	abs_prod within threshold: SAFE	-

Bước	Mô tả / Biến	Giá trị
1	Read inputs a, b, threshold	-
-	a	-15
-	b	7
-	threshold	100
2	Compute static_cast<long long>(a) * b	-
-	prod	-105
-	abs_prod	105
3	abs_prod exceeds threshold: OVERFLOW	-

Lời Giải Chi Tiết Chương 3: Bảng Biến Thiên Dãy Collatz

Lời giải & Đáp án: Bài 3.1 - Bảng Biến Thiên Dãy Collatz			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	1	0 1	N = 1 đã thỏa mãn điều kiện dừng, 0 bước
2	2	1 2	N = 2 chẵn chia đôi về 1, 1 bước
3	4	2 4	N = 4 chia đôi 2 lần về 1, 2 bước
4	8	3 8	Lũy thừa của 2: 8 -> 4 -> 2 -> 1, 3 bước
5	3	7 16	Số lẻ nhỏ: 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
6	5	5 16	5 -> 16 -> 8 -> 4 -> 2 -> 1
7	6	8 16	6 -> 3 -> 10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1
8	7	16 52	Số lẻ đạt đỉnh 52 sau 16 bước
9	9	19 52	9 -> 28 -> 14 -> 7, 19 bước đạt đỉnh 52
10	10	6 16	10 -> 5 -> 16 -> 8 -> 4 -> 2 -> 1

Bảng Vết Thực Thi Từng Bước (Chương 3)

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	1
-	max_val	1

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	2
-	max_val	2
1	Collatz iteration	-
-	current	1
-	max_val	2

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	4
-	max_val	4
1	Collatz iteration	-
-	current	2
-	max_val	4
2	Collatz iteration	-
-	current	1
-	max_val	4

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	8
-	max_val	8
1	Collatz iteration	-
-	current	4
-	max_val	8
2	Collatz iteration	-
-	current	2
-	max_val	8
3	Collatz iteration	-
-	current	1
-	max_val	8

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	3
-	max_val	3
1	Collatz iteration	-
-	current	10
-	max_val	10
2	Collatz iteration	-
-	current	5
-	max_val	10
3	Collatz iteration	-
-	current	16
-	max_val	16
4	Collatz iteration	-
-	current	8
-	max_val	16
5	Collatz iteration	-
-	current	4
-	max_val	16
6	Collatz iteration	-
-	current	2
-	max_val	16
7	Collatz iteration	-
-	current	1
-	max_val	16

Bước	Mô tả / Biến	Giá trị
0	Initial state	-

Bước	Mô tả / Biến	Giá trị
-	current	5
-	max_val	5
1	Collatz iteration	-
-	current	16
-	max_val	16
2	Collatz iteration	-
-	current	8
-	max_val	16
3	Collatz iteration	-
-	current	4
-	max_val	16
4	Collatz iteration	-
-	current	2
-	max_val	16
5	Collatz iteration	-
-	current	1
-	max_val	16

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	6
-	max_val	6
1	Collatz iteration	-
-	current	3
-	max_val	6
2	Collatz iteration	-
-	current	10
-	max_val	10
3	Collatz iteration	-
-	current	5
-	max_val	10
4	Collatz iteration	-
-	current	16
-	max_val	16
5	Collatz iteration	-
-	current	8
-	max_val	16
6	Collatz iteration	-
-	current	4
-	max_val	16
7	Collatz iteration	-

Bước	Mô tả / Biến	Giá trị
-	current	2
-	max_val	16
8	Collatz iteration	-
-	current	1
-	max_val	16

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	7
-	max_val	7
1	Collatz iteration	-
-	current	22
-	max_val	22
2	Collatz iteration	-
-	current	11
-	max_val	22
3	Collatz iteration	-
-	current	34
-	max_val	34
4	Collatz iteration	-
-	current	17
-	max_val	34
5	Collatz iteration	-
-	current	52
-	max_val	52
6	Collatz iteration	-
-	current	26
-	max_val	52
7	Collatz iteration	-
-	current	13
-	max_val	52
8	Collatz iteration	-
-	current	40
-	max_val	52
9	Collatz iteration	-
-	current	20
-	max_val	52
10	Collatz iteration	-
-	current	10
-	max_val	52
11	Collatz iteration	-

Bước	Mô tả / Biến	Giá trị
-	current	5
-	max_val	52
12	Collatz iteration	-
-	current	16
-	max_val	52
13	Collatz iteration	-
-	current	8
-	max_val	52
14	Collatz iteration	-
-	current	4
-	max_val	52
15	Collatz iteration	-
-	current	2
-	max_val	52
16	Collatz iteration	-
-	current	1
-	max_val	52

Bước	Mô tả / Biến	Giá trị
0	Initial state	-
-	current	9
-	max_val	9
1	Collatz iteration	-
-	current	28
-	max_val	28
2	Collatz iteration	-
-	current	14
-	max_val	28
3	Collatz iteration	-
-	current	7
-	max_val	28
4	Collatz iteration	-
-	current	22
-	max_val	28
5	Collatz iteration	-
-	current	11
-	max_val	28
6	Collatz iteration	-
-	current	34
-	max_val	34
7	Collatz iteration	-

Bước	Mô tả / Biến	Giá trị
-	current	17
-	max_val	34
8	Collatz iteration	-
-	current	52
-	max_val	52
9	Collatz iteration	-
-	current	26
-	max_val	52
10	Collatz iteration	-
-	current	13
-	max_val	52
11	Collatz iteration	-
-	current	40
-	max_val	52
12	Collatz iteration	-
-	current	20
-	max_val	52
13	Collatz iteration	-
-	current	10
-	max_val	52
14	Collatz iteration	-
-	current	5
-	max_val	52
15	Collatz iteration	-
-	current	16
-	max_val	52
16	Collatz iteration	-
-	current	8
-	max_val	52
17	Collatz iteration	-
-	current	4
-	max_val	52
18	Collatz iteration	-
-	current	2
-	max_val	52
19	Collatz iteration	-
-	current	1
-	max_val	52

Bước	Mô tả / Biến	Giá trị
0	Initial state	-

Bước	Mô tả / Biến	Giá trị
-	current	10
-	max_val	10
1	Collatz iteration	-
-	current	5
-	max_val	10
2	Collatz iteration	-
-	current	16
-	max_val	16
3	Collatz iteration	-
-	current	8
-	max_val	16
4	Collatz iteration	-
-	current	4
-	max_val	16
5	Collatz iteration	-
-	current	2
-	max_val	16
6	Collatz iteration	-
-	current	1
-	max_val	16

Lời Giải Chi Tiết Chương 4: Thống Kê Mảng & Chuỗi Đối Xứng

Lời giải & Đáp án: Bài 4.1 - Thống Kê Mảng & Chuỗi Đối Xứng			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	3 1 2 3 aba	1 YES	Mảng [1,2,3] avg=2.0 có 1 số > 2; chuỗi aba đối xứng
2	4 2 4 6 8 noon	2 YES	Mảng [2,4,6,8] avg=5.0 có 2 số > 5; chuỗi noon đối xứng
3	3 5 5 5 cat	0 NO	Các phần tử bằng nhau nên không có số nào > avg; cat không đối xứng
4	1 10 z	0 YES	Mảng 1 phần tử và chuỗi 1 ký tự luôn đối xứng
5	2 1 9 no	1 NO	Mảng 2 phần tử và chuỗi 2 ký tự khác nhau
6	2 4 4 gg	0 YES	Mảng 2 phần tử bằng nhau và chuỗi gg đối xứng
7	5 -10 0 10 20 30 racecar	2 YES	Có số âm, avg=10.0, phần tử > 10 là 20 và 30; racecar đối xứng
8	4 -5 -5 -5 -1 abba	1 YES	Số âm, sum=-16, avg=-4.0, số -1 > -4; abba đối xứng
9	5 1 3 5 7 9 radar	2 YES	Cấp số cộng lẻ avg=5.0; radar đối xứng
10	6 2 2 2 8 8 8 level	3 YES	Tổng 30 chia 6 = 5.0, có 3 số 8 > 5; level đối xứng

Bảng Vết Thực Thi Từng Bước (Chương 4)

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(1, 2, 3)
-	sum	6
-	avg	2
-	count_above	1
2	Palindrome checked	-
-	is_pal	true

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(2, 4, 6, 8)
-	sum	20
-	avg	5
-	count_above	2
2	Palindrome checked	-
-	is_pal	true

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(5, 5, 5)
-	sum	15
-	avg	5
-	count_above	0
2	Palindrome checked	-
-	is_pal	false

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(10,)
-	sum	10
-	avg	10
-	count_above	0
2	Palindrome checked	-
-	is_pal	true

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(1, 9)
-	sum	10
-	avg	5
-	count_above	1
2	Palindrome checked	-
-	is_pal	false

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(4, 4)
-	sum	8
-	avg	4
-	count_above	0
2	Palindrome checked	-

Bước	Mô tả / Biến	Giá trị
-	is_pal	true

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(-10, 0, 10, 20, 30)
-	sum	50
-	avg	10
-	count_above	2
2	Palindrome checked	-
-	is_pal	true

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(-5, -5, -5, -1)
-	sum	-16
-	avg	-4
-	count_above	1
2	Palindrome checked	-
-	is_pal	true

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(1, 3, 5, 7, 9)
-	sum	25
-	avg	5
-	count_above	2
2	Palindrome checked	-
-	is_pal	true

Bước	Mô tả / Biến	Giá trị
1	Array loaded	-
-	a	(2, 2, 2, 8, 8, 8)
-	sum	30
-	avg	5
-	count_above	3
2	Palindrome checked	-
-	is_pal	true

Lời Giải Chi Tiết Chương 5: Tìm Cực Trị Bằng Con Trỏ & Khoảng Cách Ô Nhớ

Lời giải & Đáp án: Bài 5.1 - Con Trỏ & Khoảng Cách Ô Nhớ			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	3 10 20 30	10 30 2	Min ở chỉ số 0, Max ở chỉ số 2, độ lệch 2
2	3 30 20 10	10 30 -2	Min ở chỉ số 2, Max ở chỉ số 0, độ lệch -2
3	4 5 1 9 3	1 9 1	Min=1 ở 1, Max=9 ở 2, độ lệch 2 - 1 = 1
4	1 42	42 42 0	Mảng 1 phần tử, độ lệch 0
5	2 7 7	7 7 0	Hai phần tử bằng nhau, cùng lưu vị trí 0
6	2 -10 10	-10 10 1	Phần tử âm và dương, độ lệch 1
7	5 -8 -3 -15 0 -2	-15 0 1	Min=-15 ở vị trí 2, Max=0 ở vị trí 3, độ lệch 3 - 2 = 1
8	4 4 2 2 4	2 4 -1	Min đầu tiên ở 1, Max đầu tiên ở 0, độ lệch 0 - 1 = -1
9	6 12 5 28 3 19 28	3 28 -1	Min=3 ở 3, Max=28 đầu tiên ở 2, độ lệch 2 - 3 = -1
10	5 -5 -2 -9 -1 -4	-9 -1 1	Toàn số âm: Min=-9 ở 2, Max=-1 ở 3, độ lệch 3 - 2 = 1

Bảng Vết Thực Thi Từng Bước (Chương 5)

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(10, 20, 30)
2	Pointer scan step	-
-	val	20
-	min_val	10
-	max_val	20
3	Pointer scan step	-
-	val	30
-	min_val	10
-	max_val	30
4	Result computed	-
-	offset	2

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-

Bước	Mô tả / Biến	Giá trị
-	a	(30, 20, 10)
2	Pointer scan step	-
-	val	20
-	min_val	20
-	max_val	30
3	Pointer scan step	-
-	val	10
-	min_val	10
-	max_val	30
4	Result computed	-
-	offset	-2

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(5, 1, 9, 3)
2	Pointer scan step	-
-	val	1
-	min_val	1
-	max_val	5
3	Pointer scan step	-
-	val	9
-	min_val	1
-	max_val	9
4	Pointer scan step	-
-	val	3
-	min_val	1
-	max_val	9
5	Result computed	-
-	offset	1

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(42,)
2	Result computed	-
-	offset	0

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(7, 7)
2	Pointer scan step	-
-	val	7
-	min_val	7

Bước	Mô tả / Biến	Giá trị
-	max_val	7
3	Result computed	-
-	offset	0

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(-10, 10)
2	Pointer scan step	-
-	val	10
-	min_val	-10
-	max_val	10
3	Result computed	-
-	offset	1

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(-8, -3, -15, 0, -2)
2	Pointer scan step	-
-	val	-3
-	min_val	-8
-	max_val	-3
3	Pointer scan step	-
-	val	-15
-	min_val	-15
-	max_val	-3
4	Pointer scan step	-
-	val	0
-	min_val	-15
-	max_val	0
5	Pointer scan step	-
-	val	-2
-	min_val	-15
-	max_val	0
6	Result computed	-
-	offset	1

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(4, 2, 2, 4)
2	Pointer scan step	-
-	val	2
-	min_val	2

Bước	Mô tả / Biến	Giá trị
-	max_val	4
3	Pointer scan step	-
-	val	2
-	min_val	2
-	max_val	4
4	Pointer scan step	-
-	val	4
-	min_val	2
-	max_val	4
5	Result computed	-
-	offset	-1

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(12, 5, 28, 3, 19, 28)
2	Pointer scan step	-
-	val	5
-	min_val	5
-	max_val	12
3	Pointer scan step	-
-	val	28
-	min_val	5
-	max_val	28
4	Pointer scan step	-
-	val	3
-	min_val	3
-	max_val	28
5	Pointer scan step	-
-	val	19
-	min_val	3
-	max_val	28
6	Pointer scan step	-
-	val	28
-	min_val	3
-	max_val	28
7	Result computed	-
-	offset	-1

Bước	Mô tả / Biến	Giá trị
1	Array loaded via pointer base	-
-	a	(-5, -2, -9, -1, -4)

Bước	Mô tả / Biến	Giá trị
2	Pointer scan step	-
-	val	-2
-	min_val	-5
-	max_val	-2
3	Pointer scan step	-
-	val	-9
-	min_val	-9
-	max_val	-2
4	Pointer scan step	-
-	val	-1
-	min_val	-9
-	max_val	-1
5	Pointer scan step	-
-	val	-4
-	min_val	-9
-	max_val	-1
6	Result computed	-
-	offset	1

Lời Giải Chi Tiết Chương 6: Khung Ngăn Xếp Đệ Quy & Ước Số Chung Lớn Nhất

Lời giải & Đáp án: Bài 6.1 - Call Stack & Đệ Quy Euclid			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	12 8	4 3	gcd(12,8) qua 3 khung ngăn xếp: (12,8) -> (8,4) -> (4,0)
2	10 5	5 2	gcd(10,5) chia hết ngay: (10,5) -> (5,0), 2 khung
3	7 7	7 2	Hai số bằng nhau: (7,7) -> (7,0), 2 khung
4	13 0	13 1	B bằng 0 chạm ngay điểm dừng base case, 1 khung
5	1 1	1 2	Hai số đơn vị: (1,1) -> (1,0), 2 khung
6	1 25	1 3	A < B hoán vị qua dư: (1,25) -> (25,1) -> (1,0), 3 khung
7	18 12	6 3	gcd(18,12) -> (12,6) -> (6,0), 3 khung
8	21 14	7 3	gcd(21,14) -> (14,7) -> (7,0), 3 khung
9	34 21	1 8	Dãy Fibonacci liên tiếp: 34 và 21 đòi hỏi 8 khung ngăn xếp
10	48 18	6 4	gcd(48,18) -> (18,12) -> (12,6) -> (6,0), 4 khung

Bảng Vết Thực Thi Từng Bước (Chương 6)

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	12
-	b	8
-	depth	1
2	Enter gcd frame	-
-	a	8
-	b	4
-	depth	2
3	Enter gcd frame	-
-	a	4
-	b	0
-	depth	3
3	Base case reached	-
99	Finished gcd	-
-	res	4
-	max_depth	3

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	10
-	b	5

Bước	Mô tả / Biến	Giá trị
-	depth	1
2	Enter gcd frame	-
-	a	5
-	b	0
-	depth	2
2	Base case reached	-
99	Finished gcd	-
-	res	5
-	max_depth	2

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	7
-	b	7
-	depth	1
2	Enter gcd frame	-
-	a	7
-	b	0
-	depth	2
2	Base case reached	-
99	Finished gcd	-
-	res	7
-	max_depth	2

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	13
-	b	0
-	depth	1
1	Base case reached	-
99	Finished gcd	-
-	res	13
-	max_depth	1

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	1
-	b	1
-	depth	1
2	Enter gcd frame	-
-	a	1
-	b	0

Bước	Mô tả / Biến	Giá trị
-	depth	2
2	Base case reached	-
99	Finished gcd	-
-	res	1
-	max_depth	2

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	1
-	b	25
-	depth	1
2	Enter gcd frame	-
-	a	25
-	b	1
-	depth	2
3	Enter gcd frame	-
-	a	1
-	b	0
-	depth	3
3	Base case reached	-
99	Finished gcd	-
-	res	1
-	max_depth	3

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	18
-	b	12
-	depth	1
2	Enter gcd frame	-
-	a	12
-	b	6
-	depth	2
3	Enter gcd frame	-
-	a	6
-	b	0
-	depth	3
3	Base case reached	-
99	Finished gcd	-
-	res	6
-	max_depth	3

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	21
-	b	14
-	depth	1
2	Enter gcd frame	-
-	a	14
-	b	7
-	depth	2
3	Enter gcd frame	-
-	a	7
-	b	0
-	depth	3
3	Base case reached	-
99	Finished gcd	-
-	res	7
-	max_depth	3

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	34
-	b	21
-	depth	1
2	Enter gcd frame	-
-	a	21
-	b	13
-	depth	2
3	Enter gcd frame	-
-	a	13
-	b	8
-	depth	3
4	Enter gcd frame	-
-	a	8
-	b	5
-	depth	4
5	Enter gcd frame	-
-	a	5
-	b	3
-	depth	5
6	Enter gcd frame	-
-	a	3
-	b	2

Bước	Mô tả / Biến	Giá trị
-	depth	6
7	Enter gcd frame	-
-	a	2
-	b	1
-	depth	7
8	Enter gcd frame	-
-	a	1
-	b	0
-	depth	8
8	Base case reached	-
99	Finished gcd	-
-	res	1
-	max_depth	8

Bước	Mô tả / Biến	Giá trị
1	Enter gcd frame	-
-	a	48
-	b	18
-	depth	1
2	Enter gcd frame	-
-	a	18
-	b	12
-	depth	2
3	Enter gcd frame	-
-	a	12
-	b	6
-	depth	3
4	Enter gcd frame	-
-	a	6
-	b	0
-	depth	4
4	Base case reached	-
99	Finished gcd	-
-	res	6
-	max_depth	4

Lời Giải Chi Tiết Chương 7: Sắp Xếp Tọa Độ Điểm Bằng Struct

Lời giải & Đáp án: Bài 7.1 - Sắp Xếp Tọa Độ Điểm			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	2 1 2 3 2 1 1	2 1	Điểm 2 có Manhattan 2 < điểm 1 có Manhattan 5
2	3 1 0 5 2 1 2 3 0 1	3 2 1	Khoảng cách Manhattan tăng dần: 1, 3, 5
3	2 1 3 0 2 0 3	2 1	Cùng Manhattan = 3, điểm 2 có hoành độ x=0 < x=3
4	1 99 0 0	99	Mảng 1 điểm duy nhất
5	2 1 0 0 2 0 0	1 2	Trùng tọa độ hoàn toàn, so sánh theo id
6	3 1 -2 0 2 2 0 3 0 2	1 3 2	Cùng Manhattan = 2, so sánh theo hoành độ: -2 < 0 < 2
7	4 1 1 1 2 -1 -1 3 -1 1 4 1 -1	2 3 1 4	Cùng Manhattan = 2, x=-1 (id 2, 3) đứng trước x=1 (id 1, 4)
8	3 10 0 4 20 2 2 30 -4 0	30 10 20	Cùng khoảng cách 4, hoành độ: -4 (id 30), 0 (id 10), 2 (id 20)
9	5 1 3 3 2 1 1 3 2 2 4 0 2 5 1 0	5 4 2 3 1	5 điểm hỗn hợp: Manhattan 1 (id 5), Manhattan 2 (id 4 x=0, id 2 x=1), Manhattan 4 (id 3), Manhattan 6 (id 1)
10	4 4 5 5 3 -2 -3 2 0 1 1 -1 0	1 2 3 4	Thứ tự sắp xếp theo Manhattan: 1 (d=1, x=-1), 2 (d=1, x=0), 3 (d=5), 4 (d=10)

Bảng Vết Thực Thi Từng Bước (Chương 7)

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-

Bước	Mô tả / Biến	Giá trị
2	Sorted points	-
3	Output point	-
-	id	2
-	dist	2
4	Output point	-
-	id	1
-	dist	5

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	3
-	dist	1
4	Output point	-
-	id	2
-	dist	3
5	Output point	-
-	id	1
-	dist	5

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	2
-	dist	3
4	Output point	-
-	id	1
-	dist	3

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	99
-	dist	0

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-

Bước	Mô tả / Biến	Giá trị
-	id	1
-	dist	0
4	Output point	-
-	id	2
-	dist	0

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	1
-	dist	2
4	Output point	-
-	id	3
-	dist	2
5	Output point	-
-	id	2
-	dist	2

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	2
-	dist	2
4	Output point	-
-	id	3
-	dist	2
5	Output point	-
-	id	1
-	dist	2
6	Output point	-
-	id	4
-	dist	2

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	30
-	dist	4
4	Output point	-

Bước	Mô tả / Biến	Giá trị
-	id	10
-	dist	4
5	Output point	-
-	id	20
-	dist	4

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	5
-	dist	1
4	Output point	-
-	id	4
-	dist	2
5	Output point	-
-	id	2
-	dist	2
6	Output point	-
-	id	3
-	dist	4
7	Output point	-
-	id	1
-	dist	6

Bước	Mô tả / Biến	Giá trị
1	Loaded points	-
2	Sorted points	-
3	Output point	-
-	id	1
-	dist	1
4	Output point	-
-	id	2
-	dist	1
5	Output point	-
-	id	3
-	dist	5
6	Output point	-
-	id	4
-	dist	10

Lời Giải Chi Tiết Chương 8: Đếm Thao Tác Vòng Lặp & Đánh Giá Giới Hạn

Lời giải & Đáp án: Bài 8.1 - Đếm Thao Tác Vòng Lặp			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	2 1 2	1 0 50	Mảng đã tăng dần, 1 phép so sánh, 0 nghịch thế
2	2 2 1	1 1 50	Mảng giảm dần, 1 phép so sánh, 1 nghịch thế
3	3 1 2 3	3 0 50	N=3 tăng dần, 3 phép so sánh, 0 nghịch thế
4	1 5	0 0 50	N=1 không có cặp so sánh nào
5	3 3 2 1	3 3 50	N=3 giảm dần hoàn toàn, nghịch thế cực đại là 3
6	3 2 2 2	3 0 50	Tất cả phần tử bằng nhau, không có cặp nào $a[i] > a[j]$
7	4 4 3 2 1	6 6 50	N=4 nghịch thế cực đại: $4*(3)/2 = 6$ phép so sánh, 6 nghịch thế
8	4 1 3 2 4	6 1 50	N=4 chỉ có 1 nghịch thế (3, 2)
9	5 2 4 1 3 5	10 3 50	N=5 có 10 cặp so sánh, 3 nghịch thế: (2,1), (4,1), (4,3)
10	5 5 1 2 3 4	10 4 50	Phần tử đầu tiên lớn nhất tạo 4 nghịch thế

Bảng Vết Thực Thi Từng Bước (Chương 8)

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	1
-	inv	0

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	1
-	inv	1

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	3
-	inv	0

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	0
-	inv	0

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	3
-	inv	3

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	3
-	inv	0

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	6
-	inv	6

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	6
-	inv	1

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	10
-	inv	3

Bước	Mô tả / Biến	Giá trị
1	Counting complete	-
-	ops	10
-	inv	4

Lời Giải Chi Tiết Bài Tập Tích Hợp: Quản Lý Hồ Sơ Thí Sinh

Lời giải & Đáp án: Bài Tập Tích Hợp 1 - Quản Lý Hồ Sơ Thí Sinh			
Test	Đầu vào (Input)	Đáp án chính thức	Ghi chú / Phương pháp
1	2 1 90 15 2 80 16	1 2 1 90 1	Điểm 90 > 80, 1 phép so sánh
2	3 1 70 16 2 95 15 3 85 17	2 3 1 2 95 3	Thứ tự điểm 95, 85, 70 ứng với id 2, 3, 1; 3 phép so sánh
3	2 1 85 16 2 85 15	2 1 2 85 1	Bảng điểm 85, thí sinh 2 trẻ hơn (15 < 16) xếp trước
4	1 42 100 18	42 42 100 0	1 thí sinh duy nhất, 0 phép so sánh
5	2 1 80 15 2 80 15	1 2 1 80 1	Cùng điểm cùng tuổi, ưu tiên id nhỏ hơn
6	3 10 50 14 20 50 14 30 50 14	10 20 30 10 50 3	3 thí sinh cùng điểm và tuổi, sắp xếp theo thứ tự id ban đầu
7	4 1 90 17 2 90 16 3 95 18 4 95 15	4 3 2 1 4 95 6	Điểm 95: id 4 (15t) rồi id 3 (18t); Điểm 90: id 2 (16t) rồi id 1 (17t); 6 phép so sánh
8	3 1 60 19 2 70 18 3 60 17	2 3 1 2 70 3	Điểm 70 (id 2), điểm 60 tuổi 17 (id 3) rồi tuổi 19 (id 1)
9	5 1 50 15 2 80 16 3 65 14 4 90 17 5 80 15	4 5 2 3 1 4 90 10	N=5 có 10 phép so sánh; sắp xếp: 90 (4), 80-15t (5), 80-16t (2), 65 (3), 50 (1)
10	4 10 100 20 20 100 18 30 100 19 40 100 18	20 40 30 10 20 100 6	Cùng điểm 100; tuổi 18: id 20 rồi 40; tuổi 19: id 30; tuổi 20: id 10

Bảng Vết Thực Thi Từng Bước (Bài Tập Tích Hợp)

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-

Bước	Mô tả / Biến	Giá trị
2	Pointers sorted	-
-	comparisons	1

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	3

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	1

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	0

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	1

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	3

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	6

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	3

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	10

Bước	Mô tả / Biến	Giá trị
1	Students initialized and pointers bound	-
2	Pointers sorted	-
-	comparisons	6